Context compaction for small language models is a content-shape adapter: a falsification study

Ahmet Barış Günaydın *Independent researcher · github.com/abgnydn/butterfly*

Abstract

A recurring idea for managing a small language model's limited context window is **tag-and-rebuild compaction**: classify each message of a conversation as `keep` / `summarize` / `melt`, rebuild the survivors into a tight token budget (the "chrysalis"), inject fresh noise, and repeat across generations. The advertised claim is that this preserves load-bearing facts better than naive recency truncation (`lastN`) under compounding noise. We pre-registered that claim, **refuted our own first pre-registration**, filed a harder regime that **confirmed**, then spent the rest of the study trying to break the confirmation. The result is narrower and more useful than the original claim: the compaction mechanism is a **content-shape adapter, not a universal context manager — the engineering work lives in the tagger, not the rebuild step.** A regex tagger and two classifiers trained on its labels (45 and 2,307 parameters) reproduce the win exactly; frontier instruction-tuned LLMs prompted to "find important messages" (qwen3-14b, gemma) do not. On adversarial transcripts whose needles are real prose rather than identifier shapes, **every tagger fails**, and the multi-generation noise injection acts as a feedback loop that locks the protocol onto noise once the needle is dropped in the first cocoon. Validated against the external **LongMemEval** benchmark (ICLR 2025), the directional finding holds across three orders of magnitude of context size (6.6K to 1.25M tokens): a 14-parameter classifier trained on representative labels preserves 86% of evidence turns at 3× compression on the oracle split and beats `lastN` by +16–64pp everywhere — but the same classifier is **domain-locked** (0% on engineering chat, whose needles it learned to ignore), and a downstream LLM-judged QA test shows the mechanism delivers a real 2.5× lift over `lastN` on modest haystacks (38% vs 15%) yet **collapses to 0–5% on large multi-session haystacks** where the bottleneck is retrieval, not compaction. We report the two refutations, the confirmation, and the boundary, and we are explicit that the chrysalis loop — the original novel claim — was never validated downstream.

---

1. Introduction

Small, local language models run into their context window quickly, and the obvious fix — drop the oldest tokens (`lastN` truncation) — discards load-bearing facts when they happen to be old. A more selective alternative is to *tag* each message by importance and *rebuild* a compacted context from the survivors, iterating as new content arrives. This is intuitively appealing and easy to demo: keep the file paths and decisions, melt the small talk, watch the important content survive.

Intuition and demos are not evidence. This study treats the compaction mechanism as a falsifiable claim and runs it through a pre-registration discipline: each prediction is filed with a numeric confirm/refute threshold *before* the experiment, and the outcome is recorded regardless of which way it goes. The pre-registrations were filed in the git-timestamped predictions log of the neuropulse visualizer, inside whose browser demo the mechanism first ran (entries `P-20260512-05`, `P-20260515-06`).

The contribution is not a new memory architecture. It is a map of *when* tag-and-rebuild compaction beats truncation and *why* — located by refuting the broad version of the claim twice, confirming a narrow version, and then probing the narrow version against an external benchmark until its boundaries are visible. The headline is that the mechanism's power is entirely in the tagger's prior, and that prior must match the load-bearing-content distribution of the deployment domain. There is no universal butterfly.

2. Methods

2.1 The mechanism

A conversation is a list of messages. A **tagger** assigns each message a label in {keep, summarize, melt}. The **chrysalis** rebuild concatenates kept messages verbatim and the first sentence of summarized messages, dropping melted ones, truncated to a fixed token budget. One **generation** is a tag-then-rebuild pass; between generations, off-topic noise messages are injected. The original demo ran N_GENERATIONS = 3. The evaluation question is whether the planted load-bearing fact (the "needle") survives in the final compacted context better than under a lastN truncation of the same conversation at the same budget.

2.2 Taggers

Five taggers were tested under matched protocol: (i) a hand-tuned **regex** tagger firing on identifier shapes (file paths, ticket IDs, channel names, package mentions, line ranges, decision markers tagged keep; bare acknowledgements and short tangents tagged melt); (ii) a **14-feature softmax classifier** (45 parameters) trained on the regex's own labels; (iii) a **768-dimensional embedding + linear head** (2,307 parameters, nomic-embed features) trained the same way; (iv) **frontier instruction-tuned LLM taggers** (qwen3-14b, gemma-4) prompted to select important messages, across several prompt and output-format strategies; and (v) a **14-feature classifier trained on the LongMemEval oracle's own has_answer labels** (~11K labeled turns).

2.3 Scoring and datasets

The pure-code experiments score *needle-keyword coverage* — substring match of the load-bearing identifiers in each transcript's expected fact against the compacted context — which needs no LLM judge and is deterministic. External validation uses **LongMemEval** (Wu et al., ICLR 2025), the canonical long-term-conversational-memory benchmark, whose evidence turns are flagged has_answer: true; the direct metric is the fraction of evidence turns that survive compaction (*turn-rate*), with a secondary *answer-string-in-memory* metric. The three splits span the scale axis: oracle (~6.6K tokens, evidence local), s (~121K tokens, 2 evidence turns in 550), and m (~1.25M tokens, 1 evidence turn in 5,057). The downstream test feeds the compacted memory plus the question to an answerer (qwen3-4b) and grades the answer with two independent LLM judges (qwen3-4b and gemma-4) to control for same-model bias.

3. Results

3.1 Two pre-registrations: refuted, then confirmed

P–20260512–05 (filed): at the original demo regime — 12-message transcripts, 400-token budget, 1 generation — needle preservation should beat lastN by ≥ 15pp across four transcripts. **Refuted.** lastN tied butterfly on two transcripts and beat it on the other two. At a generous budget on short transcripts, recency truncation already captures the needle; the mechanism has nothing to differentiate. (The original Phi-3-in-browser harness could not gather the pre-registered sample under tab-throttling — six attempts, one completed run — so the protocol was re-run in pure code with a regex tagger, 4 ms total, same transcripts and scoring.)

P–20260515–06 (filed): file a regime where compaction must matter — 38-message transcripts (melt-able padding around the core), 100-token budget, 3 generations with fresh noise each round. **Confirmed.** All four transcripts: butterfly preserves 100% of needle keywords, lastN 0%, mean Δ = 100pp.

3.2 The phase boundary

Two binary outcomes do not say *where* the mechanism stops mattering. A sweep of 8 budgets × 6 lengths × 5 generation counts × 4 transcripts (960 cells) locates a roughly diagonal boundary: butterfly stops winning once the budget exceeds ~30–40% of the original transcript size, because lastN then already fits

the needle; more generations expand the win region. The refuted point sits in the tie zone, the confirmed point deep in the win zone — both are samples from a real phase structure, not luck.

3.3 Swapping the tagger: the win lives in the prior

The natural objection is that the regex is hand-fit to four transcripts. It is not the source of the win. Two classifiers trained on the regex's labels reproduce it exactly; a frontier LLM tagger does not.

tagger	parameters	hard-regime result
regex (hand-tuned)	~14 rules	**100/0, Δ=100pp**
14-feature softmax	45	**100/0, Δ=100pp**
768-dim embed + linear	2,307	**100/0, Δ=100pp**
qwen3-14b, one-char output, cap=3	frontier	0/0, Δ=0pp

The LLM over-tags (52–63% of messages marked `keep` vs the regex's ~8%), bloats the chrysalis past budget, and on JSON-format failures falls back to a regex that cannot read its own rebuilt string. Four further LLM configurations (output shape, keep-caps, a smaller model, identifier-first prompts) all failed: with a hard cap the LLM picks three messages, just not the needle-carrying ones. It prioritizes decision-language; the needles are literal identifier shapes. Both learned classifiers, by contrast, recover the regex's boundary by gradient descent — partly tautological (they train on regex labels) but informative: the win is in the *feature distribution the tagger weighs*, not in pathological rule code.

3.4 Adversarial transcripts: every tagger fails, and noise compounds

If the win is shape-specific, transcripts whose needles are real load-bearing prose that does *not* take identifier shapes should break every tagger. Four such transcripts (a numeric threshold in prose, a relative deadline, a stated preference, a buried causation) at the hard regime:

tagger	mean Δ over 4 adversarial transcripts
regex	0pp (every message tagged `melt`; no feature fires)
trained 14-feature	0pp (inherits regex blind spots)
embed 768-dim	0pp
qwen3-14b, one-char	0pp (picks 5 messages, none the needle)

Every tagger fails. A second finding the confirmation did not show: across all four adversarial transcripts the LLM tagger's generation-3 output is *identical* — a noise message injected at generation 2. The multi-generation loop acts as a feedback amplifier: once the needle is dropped in the first cocoon, no later generation recovers it, and the only content that survives is whatever the tagger consistently labels `keep`. If that is noise, the protocol locks onto noise.

3.5 External validation: LongMemEval across three orders of magnitude

On the `oracle` split (500 real conversations), the regex tagger wins on independent data — turn-rate 21.4–29.0% across budgets 256–2048 vs `lastN`'s 3.3–23.4%, a +5 to +18pp margin. Two effects appear immediately: (a) classifiers trained on the tiny in-domain set *overfit* (the 45-param flat at ~4.7%, the 2,307-param at 0%), and (b) different metrics have different winners — `lastN` wins answer-string-in-memory at high budgets (116/479 vs 63/479 at budget 2048) because answers often live in recent turns. A 60/40 **hybrid** of chrysalis and a `lastN` window matches or beats regex on turn-rate at budgets ≥ 512 *and* lifts answer-recall to 99/479 — best of both.

Training the 14-parameter classifier on LongMemEval's *own* `has_answer` labels reverses the overfitting result and scales:

split	avg context	longmem-trained @ best budget	vs `lastN`
oracle	6.6K tok	86.1% @ 2K (3× compression)	+63pp
s	121K tok	20.6% @ 4K (30× compression)	+17pp
m	1.25M tok	18.8% @ 32K (39× compression)	+16pp

Same 1.2 KB classifier file, three peer-reviewed benchmarks, consistent ordering (longmem-trained > longmem-hybrid > regex > `lastN`) at every budget. On `m` — one evidence turn in 5,057 — it preserves 19% of evidence at 2.5% of the original context, 7× `lastN`'s rate. The mechanism scales with the right tagger.

3.6 The boundaries: per-type loss, domain-lock, and downstream collapse

Three results fence the claim in.

Per-question-type (oracle, budget 2048, longmem-trained). The classifier wins 5 of 6 question types by 53–93pp and *loses* `single-session-assistant` by 29pp, because its dominant learned weight (`log_length` ≈ −3.25) down-weights long messages — correct when evidence is short user statements, wrong when evidence is a long assistant response. Full coverage needs question-type routing.

Cross-domain (domain-lock). The LongMemEval-trained classifier has *negative* weights on identifier features (`file_path`, `decision_kw`, `proper_name`) — the exact opposite of what engineering chat needs. On the original four engineering transcripts it preserves ~5% of needles vs the regex's 100%. The 14-feature template is rich enough to encode either prior; training data picks which. Production butterfly is therefore *one classifier per deployment domain*, not one universal tagger.

Downstream QA. Substring preservation overstates usefulness by ~2×. With an LLM answerer and two judges on the oracle split, the trained tagger reaches 38%/29% answer accuracy vs `lastN`'s 15%/7% (inter-judge agreement 87–93%, so the ranking is not a same-model artifact) — a real 2.5× lift. **Then it breaks**: on the `s` split (~50–100K-token haystacks, 50 sessions) every strategy collapses to 0–5%. Tag-by-message compaction cannot surface the right session out of fifty; that regime needs retrieval (vector DB / BM25) as a pre-step before compaction helps at all.

4. Honest limitations

- **The chrysalis loop was never validated downstream.** The QA evaluation used single-pass tag-and-rebuild. Whether iterated multi-generation compaction with noise beats single-pass at downstream QA — butterfly's *original* novel claim — remains untested. Every confirmation in this study is of the tagger, not the loop.
- **The mechanism is not a long-term memory system.** It fails on large multi-session haystacks (LongMemEval `s`, `m`) where the bottleneck is retrieval, not compaction.
- **The result is tagger-prior-specific, not tagger-agnostic.** The classifier inherits whatever its training labels prioritize; cross-domain transfer is ~0%. Per-domain labels or a multi-domain corpus are required to deploy.
- **`lastN` is a weak baseline.** Production memory systems (mem0, Letta) use vector retrieval plus LLM summarization. Beating `lastN` is necessary, not sufficient, for production relevance; those systems were not benchmarked.
- **Not a substitute for frontier `/compact`.** When a frontier model is available, its summarization is cheaper and better. The "small local classifier
  - small local answerer" lane is for the local-first niche.

5. Related work

LongMemEval (Wu et al., ICLR 2025) and MemoryAgentBench (ICLR 2026) are the external references for long-term conversational memory; this study uses LongMemEval as an independent oracle rather than

proposing a competing system. Production memory frameworks (mem0, Letta) combine vector retrieval with LLM summarization and are the real prior art for deployment; we benchmark only against fixed-budget recency truncation and are explicit (§4) that this is a weak baseline. Context compaction by importance tagging is folklore in agent tooling; the contribution here is to subject it to pre-registered falsification, to localize its power in the tagger's prior rather than the rebuild step, and to map its failure boundaries (adversarial shapes, domain-lock, large-haystack retrieval) against an external benchmark across three orders of magnitude of context size.

6. Software availability and reproducibility

Source: `github.com/abgnydn/butterfly` (MIT). The hard-regime confirmation runs in pure code with no LLM and no GPU — `node butterfly-purecode-hard.mjs`, 4 ms, deterministic. The phase sweep (`butterfly-sweep-phasediagram.mjs`), the tagger comparison (`butterfly-llm-tagger.mjs`), the trained taggers (`butterfly-train-classifier.mjs`, `butterfly-train-embed.mjs`, `butterfly-train-on-longmem.mjs`), the adversarial transcripts (`butterfly-adversarial.mjs`), the LongMemEval harness (`butterfly-longmemeval.mjs`, `butterfly-crossdomain.mjs`), and the downstream QA evaluator (`butterfly-qa-eval.mjs`) reproduce every number above; LLM-tagger and QA runs use a local LM Studio endpoint. The early-April origin harness (`experiment.mjs`, `transgen.mjs`, `hybrid.mjs`) is preserved in the same repository. Pre-registrations with their git-timestamped audit trail are in the neuropulse predictions log (`PREDICTIONS.md`, entries `P-20260512-05`, `P-20260515-06`).

Generative-AI disclosure. Portions of the software, documentation, and this manuscript were drafted with a large language model used as a coding and writing aid; all output was author-reviewed, correctness was enforced by deterministic keyword-coverage scoring and the committed per-generation traces, and every quantitative claim is traceable to a recorded run or an external benchmark.

Statements. Sole author; no competing interests; no external funding.

7. Conclusion

Tag-and-rebuild context compaction for small language models is real but narrow: it beats recency truncation at tight budgets under compounding noise **only when the tagger's prior matches the load-bearing-content distribution of the transcripts being compacted.** The engineering problem is the tagger, not the rebuild step — a 45-parameter classifier trained on a few hundred labeled examples of a domain's load-bearing shapes outperforms both hand-coded rules and zero-shot frontier-LLM tagging, and the win holds against an external benchmark from 6.6K to 1.25M tokens of context. But the same classifier is domain-locked, the mechanism fails where retrieval rather than compaction is the bottleneck, and the multi-generation loop that was its original selling point was never validated downstream. The useful deliverable is a recipe with a stated domain of validity, arrived at by refuting the broad claim before defending the narrow one.

References (to be formatted)

- **Wu et al., 2025** — "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," ICLR 2025. github.com/xiaowu0162/LongMemEval
- **MemoryAgentBench, 2026** — long-term memory agent benchmark, ICLR 2026.
- mem0; Letta (MemGPT) — production conversational-memory frameworks (prior art, not benchmarked).
- Phi-3-mini-4k-instruct (Microsoft) — the small model in the original demo.

Draft v0.1 — companion to the neuropulse visualizer (github.com/abgnydn/neuropulse), inside whose browser demo the mechanism first ran. Built from the committed harness and per-generation traces;

LLM-tagger and downstream-QA numbers come from local LM Studio runs. To be converted to LaTeX before submission to an NLP/ML-systems or agents venue.